Course : Individual Software Development Process' 2026

Iteration Report : Iteration 5

By Ultrasmooth

Chosen Irl Challenge : Project A - Lab Data Management



Jirat Kiattrairong	6810545506
Napakhet Namsrioun	6810545727
Panyasiri Aimngern	6810545743
Panisara Niyathirakul	6810545751

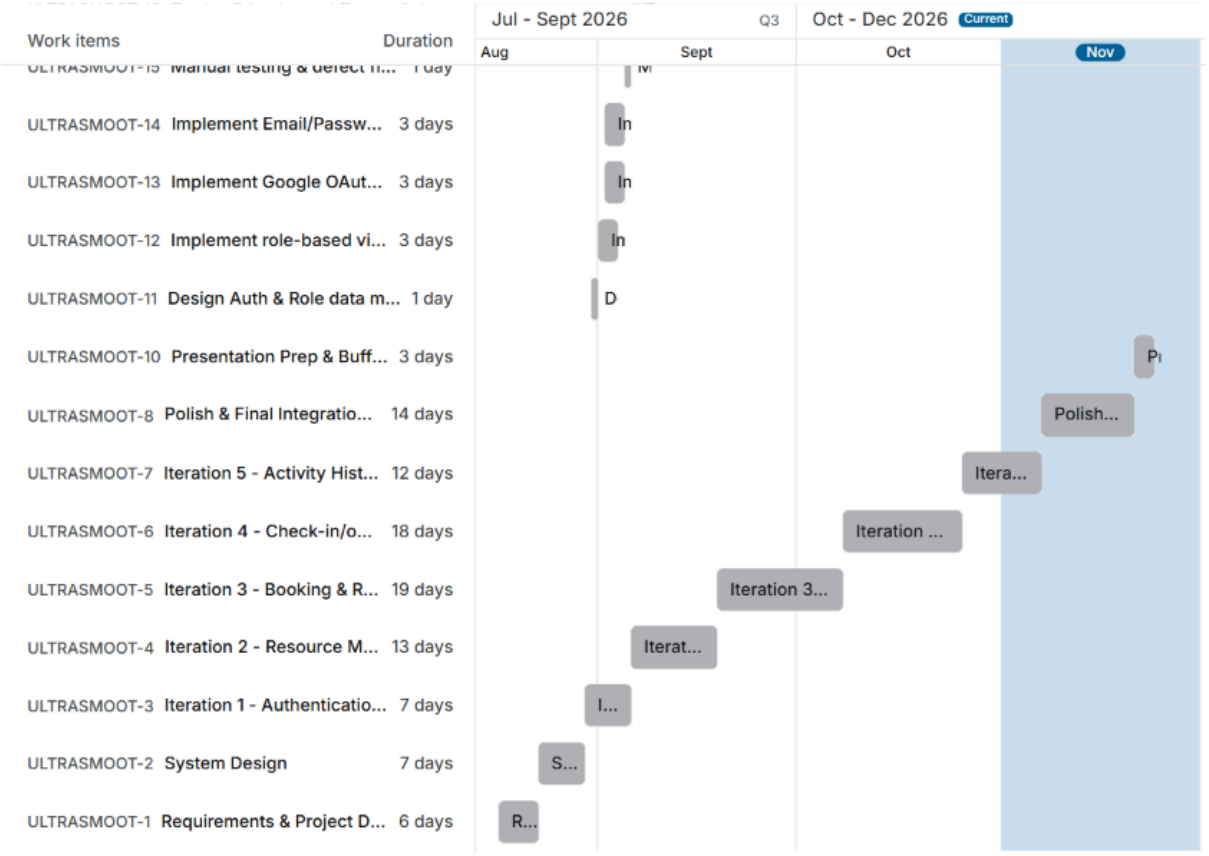
Table of Contents

1. Project Snapshot..... 1
1.1 Project-Wide Gantt Chart.....1
1.2 Current Iteration Gantt Chart.....2
2. Sprint/Board Status Snapshot.....3
3. Scope Addition/Change Log..... 6
4. Retrospective.....7
5. Testing & Quality Notes.....8
6. Project Risk Register..... 10
7. Individual Contribution Log..... 11

1. Project Snapshot

Project Repository : https://github.com/Ultrasmoot/Ultrasmooth

1.1 Project-Wide Gantt Chart

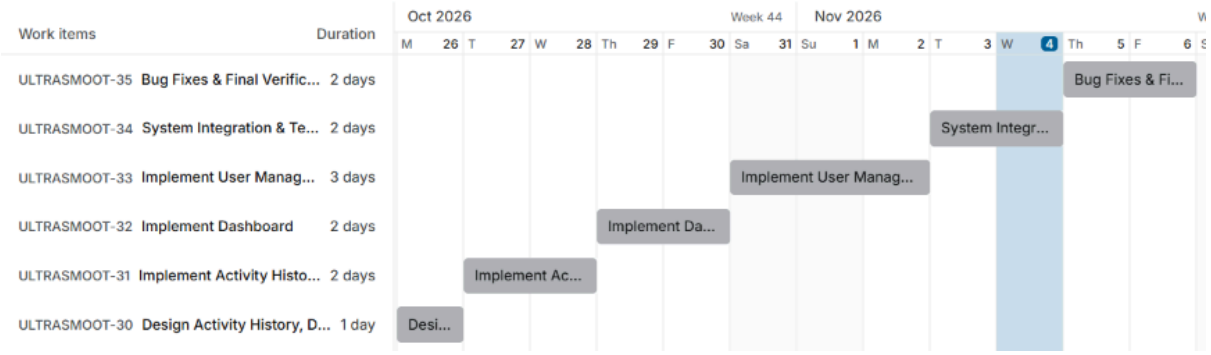


Critical Path :

Requirements & Project Documentation
↓
System Design
↓
Iteration 1 - Authentication & Access (SRS-11, SRS-12)
↓
Iteration 2 - Resource Management (SRS-1, SRS-2)
↓
Iteration 3 - Booking & Request Management (SRS-3, SRS-4, SRS-5)
↓
Iteration 4 - Check-in/Check-out & Maintenance (SRS-10, SRS-6, SRS-7)
↓
Iteration 5 - Activity Log & Dashboard & User Management (SRS-8, SRS-9, SRS-13)
↓
Final Integration & Deployment

Critical Path Duration : 99 calendar days

1.2 Current Iteration Gantt Chart



Critical Path :

Design Activity History, Dashboard & User Management
↓
Implement Activity History
↓
Implement Dashboard
↓
Implement User Management
↓
System Integration & Testing
↓
Bug Fixes & Final Verification

Critical Path Duration : 12 calendar days

2. Sprint/Board Status Snapshot

Iteration Goal : The goal of Iteration 5 was to give the lab managers control and visibility over the whole system. Every important action is now saved in an activity log, and administrators and professors can search it. They can also open a dashboard that shows resource usage and what needs attention. Administrators can manage user accounts and roles, so they decide who can use the system. This iteration supports Sub-Goals SG-4 (History & Dashboard) and SG-6 (System Access & Role Management) from our SRS, through SRS-8, SRS-9 and SRS-13. It is also the last feature iteration, so we used the end of it for integration and bug fixing.

During this iteration, the team focused on three user stories :

- US-8 : View and filter the activity history
- US-9 : View the operations dashboard
- US-13 : Manage user accounts and roles

Capacity Declared : 3 user stories (US-8, US-9 and US-13), the same as Iterations 3 and 4. We kept three because Iteration 6 was removed from the plan and US-13 had to fit here. We accepted this because US-13 is mostly separate from the other two stories : it does not share data with the dashboard and it only writes one more type of event into the log.

User Story	Description	Owner	Status	Within Capacity (Y/N)	Notes
US-8	As a Lab Administrator or Professor, I want to view and filter the activity history so I can see who did what and when.	Napakhet Namsrioun	Done	Y	Administrators and professors can view the system logs, while other roles cannot access them (SRS-8).The log records important system activities, including bookings, check-in/check-out, failed check-in codes, maintenance, resource changes, announcements, user account changes, and failed login attempts. Each record stores the actor's name at the time of the action, action type, related resource, and a short summary. Logs are saved in the same transaction as the action, so if an action fails and is rolled back, no log is created. The log is read-only through the API. Records are sorted from newest to oldest, with 20 entries per page. Users can filter by date range, user, action type, and resource. Category tabs are also available. By default, records older than 45 days are automatically deleted.
US-9	As a Lab Administrator or Professor, I want to see a dashboard so I can check the lab status and resource usage quickly.	Panyasiri Aimngern (dashboard queries, utilization calculation) Panisara Niyathirakul (dashboard controller, access check)	Done	Y	Read-only for administrators and professors (SRS-9). It shows resources by status (Available, In Use, Maintenance), active bookings, pending requests, open issues and reports waiting for review. Archived resources are not counted. Utilization per category is booked hours divided by capacity hours in the selected period, and it never goes

		Jirat Kiattrairong (Dashboard page)			above 100%. A booking that is only partly inside the period counts only the overlapping time. The period can be 7, 30 or 90 days, or a custom range of up to 366 days.
US-13	As a Lab Administrator, I want to manage user accounts and roles so I can control system access.	Panisara Niyathirakul	Done	Y	Only administrators can use these functions. The role is checked on the server, not only in the menu (SRS-13). An administrator can list users (search by name, email, student ID, faculty or major, and filter by role and status), create an account, edit it, deactivate it and reactivate it. An account created by an administrator is active and its email counts as verified. Any role can be assigned or changed, and the change works immediately. A deactivated account cannot log in with a password or with Google, and a token it already has stops working on the next request. Safety rules: an administrator cannot deactivate their own account or change their own role, and the last active administrator can never be deactivated or demoted. If the email of a Google-linked account is changed, the Google link is removed. Students need a 10-digit student ID, undergraduate students need a class year, and professors can have expertise and a short bio. Every change is saved in the activity log, and an edit with no real change is not logged. The API never returns the password hash.

3. Scope Addition/Change Log

Item added/changed	Reason	What as dropped to compensate (one-in-one-out)
Automatic check-out. If a student checked in but did not check out, the system checks them out 20 minutes after the booking ends. The resource becomes Available again, but only if it is still "In Use" and nobody else is checked in. A resource in Maintenance stays blocked. The student gets a notification and the action is saved in the log as "Automatic check-out". The 20 minutes can be changed in the settings (0 to 240).	In Iteration 4 we found that a forgotten check-out keeps a resource "In Use" and blocks the next student (risk R-07). The dashboard also needs a correct "In Use" number.	Nothing was dropped. This is a small background job outside the three declared stories.
Cancel a booking. Ph.D. and Undergraduate students can cancel their own booking while it is Pending, or Approved and not used yet. The status becomes "Cancelled" and the slot is free again. A booking that is already checked in or already ended cannot be cancelled. Administrators get a notification only when an Approved booking is cancelled.	Students could only wait or ask an administrator, so unused slots stayed blocked.	Nothing was dropped. This extends the booking flow from Iteration 3 and adds one database change (migration 007).
US-13 moved into this iteration. The plan had a separate Iteration 6 for user management. The iteration was removed and US-13 was added to Iteration 5.	The course schedule left no time for another iteration, and user management is needed before the final release.	Iteration 6 was dropped. No user story was reduced, but the project now has five iterations and the final integration starts earlier.

4. Retrospective

What went well	What didn't go well	Action items for next sprint
The log row and the notifications are saved in the same transaction as the action. A rolled-back action leaves nothing behind, and we have tests for that.	Three stories and two extra changes filled the whole iteration. There was no buffer left for review fixes.	Use the last days (integration and bug fixing, tasks 34 and 35) only for fixing and testing, not for new features.
Deactivation works at once. The check is made on every request, so we did not need a token blacklist.	The rate limiter still keeps its counters in memory.	Replace it with Redis or Flask-Limiter before deployment.
The "last administrator" rule for US-13 locks all administrator rows in the same order. Two administrators cannot demote each other at the same moment and leave the lab without an administrator.	The auto check-out job runs inside the web app. With several workers, each one runs the job. It is safe because every booking is locked and checked again, but it is not clean.	Decide before deployment: one worker, or a separate job.

5. Testing & Quality Notes

Note : The team reviewed the implementation of US-8, US-9 and US-13, and the two changes in section 3. We checked input validation, role checks on every new endpoint, transaction handling and the lock order (resource row first, then the booking; administrator rows first for user changes).

Automated tests. Run from backend/ with python -m unittest discover -s tests: 307 tests, 295 passing and 12 skipped by default. The skipped ones are the real-MySQL integration tests. They run in a throw-away MySQL container with docker compose -f docker-compose.test.yml up --build --abort-on-container-exit.

New test files for this iteration:

- **test_activity_notifications_dashboard.py** (28 tests) : log access, filters, pagination, notifications, dashboard numbers.
- **test_user_management.py** (26 tests) : access control, create, edit, roles, deactivate, last-administrator rules, log entries.
- **test_cancel_and_cleanup.py** (27 tests) : cancelling a booking, automatic check-out, and deleting old records.
- **test_integration_mysql_features.py** (3 tests) : the log, the dashboard queries and the demo seed on a real MySQL.

US-8. We tested that only administrators and professors can read the log, and that a booking flow writes who did what. A failed decision leaves no log row. A failed login is logged without the password. Filters by user, action and date work, and the last day of a range is included. Wrong filters return 400, not 500. Pages are newest first, and the log cannot be changed through the API.

US-9. We tested that only administrators and professors can open the dashboard. Archived resources are not counted. “Active” means approved and covering the current time. Open issues and reports waiting for review are counted separately. Utilization is booked hours over capacity hours, a booking partly outside the period counts only the overlap, and the value never goes above 100%. Custom periods and wrong periods are checked.

US-13. We tested every route to make sure only an administrator can use it, and that the list never contains a password hash. We tested search and the role and status filters. A new student can log in straight away, a new professor appears in the professor directory, and wrong input (name, email, role, student ID, year, duplicates, a body that is not a JSON object) is refused. Emails are saved in lower case. Changing a role works immediately, and a student who becomes an undergraduate needs a year. An administrator cannot change their own role or deactivate themselves, one administrator can demote another, and the last active administrator cannot be demoted or deactivated. A deactivated account cannot log in and its old token stops working, and reactivating gives access back. A deactivated administrator loses admin access, and a deactivated professor leaves the directory. Every change is written to the log, and an edit with no change writes nothing.

Cancel booking. We tested that a cancelled booking frees the slot and saves the cancel time, and that only the owner (a student) can cancel. Someone else's booking looks like a missing one. Cancelling is refused when the booking is checked in, ended or already decided. A cancelled booking cannot be approved or checked in. Administrators are told only about an approved booking.

Automatic check-out. We tested that a booking is closed after the 20 minutes and the resource is freed. Nothing happens inside the 20 minutes, or for bookings that were not checked in or are already checked out. A resource in Maintenance stays blocked. If another student is still checked in, the resource stays "In Use". Running the job twice does not repeat the check-out.

Manual testing. We walked through the demo seed. An administrator opened the dashboard and the activity log with filters. The administrator created a professor, changed a student's role, deactivated an account and checked that this account could not log in. We tried to deactivate our own account and to demote the only administrator, and both were refused. A student cancelled an approved booking and the administrator saw the notification. We also checked in a student and let the booking time pass, to see the automatic check-out.

```
nstead: https://pip.pypa.io/warnings/venv

[notice] A new release of pip is available: 24.0 -> 26.3
[notice] To update, run: pip install --upgrade pip
.......................................................................... [ 24%]
.......................................................................... [ 49%]
.......................................................................... [ 74%]
.......................................................................... [ 99%]
...                                                                      [100%]
307 passed in 71.35s (0:01:11)
[+] down 3/3
 ✓ Container vase-test-tests-1   Removed                                  0.1s
 ✓ Container vase-test-test-db-1 Removed                                  1.5s
 ✓ Network vase-test_default     Removed                                  0.2s
PS C:\Users\J\Documents\ISP\sprint 5\source>
```

6. Project Risk Register

Risk ID	Description	Likelihood (L/M/H)	Impact (L/M/H)	Mitigation/ Contingency	Status
R-01	Double booking or double confirmation. Row locks should stop it, but no test uses parallel requests.	L	H	All modules lock the resource row first. Add two-thread tests on the real MySQL before deployment.	Open
R-02	The rate limiter keeps counters in memory, so a restart resets them and workers do not share them.	M	M	Use Redis or Flask-Limiter before deployment. The 3-attempt check-in limit is already stored in the database.	Open
R-03	Check-in depends on SMTP. If the mail server is down, students cannot check in.	M	H	Failed returns a clear error. Consider an administrator override in the future.	Open
R-04	A student forgets to check out, so the resource stays In Use.	M	M	The system checks the student out automatically 20 minutes after the booking ends.	Closed
R-05	Old log and notification rows are deleted after 45 days, so the history is not kept for long.	M	M	The period can be changed in the settings (0 means keep everything). Ask the stakeholder how long the log must be kept.	Open
R-06	The lab could be left without an administrator, or an administrator could lock themselves out.	L	H	The server refuses self-deactivation, self-demotion and removing the last active administrator. Administrator rows are locked in a fixed order.	Closed

7. Individual Contribution Log

Student Name	Tasks owned this sprint	Course concept(s) applied	Evidence	Self-rated effort (1-5)
Napakhet Namsrioun	**US-8 (server) :** wrote the activity log on the server. The model and controller save and read the events, and services/events.py is a small helper that the other modules call when something important happens. This way the logging code is written once and not copied into every module. **US-8 (data and page) :** designed the activity_log and notifications tables (migration 004) and built the Activity log page: a timeline of event cards, filters for date, user and action type, category tabs, and a list that loads one page at a time while the person scrolls. **US-8 (tests) :** wrote test_activity_notifications_dashboard.py and the real-MySQL tests for the new features. **Extra - Failed logins :** added a call in auth_controller.py, so a failed login appears in the log. The password is never saved. **Extra - Test setup and appointments :** wrote the in-memory test database (sqlite_db.py), the throw-away MySQL setup (docker-compose.test.yml), test_security.py, test_auth_followup.py, the appointment model and its tests.	**Transaction design :** every log row is written in the same transaction as the action it describes. If the action fails and is rolled back, the log row is rolled back too, so the log never shows something that did not happen. **Audit trail design :** the log can be read through the API but not changed. Each row keeps the name of the person at that moment, so the history stays correct if an account is renamed later. **Sensitive data handling :** a failed login is logged with the email only, and the password is never stored. **Test environment isolation :** fast tests use an in-memory database, and the real-MySQL tests run in their own container with their own database.	**4c4888c** Implemented the Activity Log model to store system activity records. **2ca7a74** Implemented the Activity Log API for retrieving activity records. **3432cf2** Implemented activity log and notification events. **09168bf** Added failed login tracking to the Activity Log. **f136504** Created database tables for Activity Log, Notifications, and Appointments. **95a2472** Added the Activity Log page to the frontend. **da78755** Added tests for Activity Log, Notifications, and Dashboard features.	5

Panisara Niyathirakul	**US-13 (server) :** wrote the user management. An administrator can list and search users, create an account, edit it, deactivate it and reactivate it, and change any role. A deactivated account cannot log in, and its old token stops working on the next request. The model also blocks unsafe actions: an administrator cannot deactivate their own account or change their own role, and the last active administrator can never be deactivated or demoted. **US-13 (page and tests) :** built the User & Access page with a card for each person, a search box and role and status filters, and wrote test_user_management.py for every route and rule. **Notifications :** wrote the notification model, the controller and the bell in the top corner, which shows the unread number and lets the person mark messages as read. **Extra - Automatic check-out :** wrote services/housekeeping.py. A background job runs every minute and checks out a student 20 minutes after the booking ended if the student forgot. It also deletes old log and notification rows. The tests are in test_cancel_and_cleanup.py. **US-9 (controller) :** wrote dashboard_controller.py, which allows only administrators and professors. **Extra - Setup :** added log events to check-in, check-out and announcements, and updated docker-compose.yml, app.py and the demo seed.	**Role-based access control :** every user management route checks the role on the server. Hiding the menu item in the page is not the protection. **Business rules in the Model layer (MVC) :** the safety rules for administrators are in user_admin_model.py, and the controller only turns the result into JSON. **Lock ordering :** when an administrator is changed, all administrator rows are locked in the same order. Two administrators acting at the same moment are handled one after the other, so they cannot both remove the last administrator. **Validation at the point of use :** the automatic check-out reads the booking again under the lock. The resource becomes Available only if it is still “In Use” and nobody else is checked in. A resource in Maintenance stays blocked.	**34afc43** Implemented the User & Access management feature for administrators to manage user accounts and roles. **3f9d105** Developed the frontend for User Access management. **697f448** Added functionality for editing user accounts **77f2f92** Added functionality to activate and deactivate user accounts. **70faa46** Added role-based access control to User Access **b047082** Added the notification bell to the interface. **bde5620** Updated the check-in model to support automatic check-out. **e163e10** Added Iteration 5 application setup in app.py. **e569e38** Updated Docker Compose configuration for Iteration 5.	5

Jirat Kiattrairong	**Cancel booking (server) :** wrote the cancel method in booking_model.py and the route in booking_controller.py. A student can withdraw a Pending request, or an Approved booking that is not used yet. The status becomes “Cancelled”, the time is saved in cancelled_at, and the slot is free again. Wrote migration 007 for the new column. **Cancel booking (page) :** added the “Cancel request” window, the “Cancelled” badge, the filter chip, and the text that shows when a booking was cancelled. **US-9 (page) :** built the Dashboard page: the menu item that only administrators and professors see, the cards with the main numbers, the utilization view, the period selector and the later redesign. **Log events :** added the events for booking submitted, booking decided and the maintenance steps, so they appear in the activity log. **Extra - Settings and platform :** added the settings for the background job in config.py (grace time, interval, keeping time), worked on the rate limiter and the authentication middleware, handled time zones in the booking form, and made the new colour theme.	**State transition rules :** a booking can be cancelled only from Pending, or from Approved before check-in. The system checks the current status before saving, so a booking that was already checked in, ended or decided cannot be cancelled twice. **Deadlock avoidance :** the lock order is the resource row first and then the booking, the same order that booking, approval and check-in use. **Transaction design :** the log row for a booking or maintenance change is saved in the same transaction as the change. **Time zone handling :** the booking form sends a time with its zone, so the check-in window and the automatic check-out use the right time.	**108651d** Maintenance issue reporting and management **6138927** maintenance_model.py **934967b** maintenance_controller.py **d303abc** Maintenance guard in resource_model.py **574207c 11d1f7d** Maintenance and Report Issue UI **18f0b8a** Resource restoration functionality **da66865** Resource detail feature **e63a439 57f101f** Booking cancellation functionality **6d63f0c** Update resource status after maintenance **e80b3f3** Demo records in the seed file	5

Panyasiri Aimngern	**US-9 (queries) :** wrote dashboard_model.py. It counts resources by status, active bookings, pending requests and open issues, and calculates the utilization for each category. Utilization is booked hours divided by capacity hours in the selected period, and it never goes above 100%. Archived resources are not counted. **Extra - Resource detail page :** build the page that opens when someone clicks a resource card. Added the new optional fields to resource_model.py (migration 005) and to the edit window, and made the cards clickable without breaking the buttons on them. **Extra - Appointments :** wrote the appointment controller and migration 006. **Extra - Other :** added the pin function for announcements, wrote db.py and test_archive_resource.py, and made archive and unarchive record who did them.	**Interval overlap :** a booking that starts or ends outside the selected period counts only the hours inside it. Because of this, utilization is correct for any period, including a custom range. **SQL aggregation :** the dashboard gets its numbers from a few grouped queries, not from many small ones. **Relational database design :** the new resource details and the appointments came with migrations and foreign keys, so an existing database can be upgraded without losing data.	**44b2563** implement dashbord model **4d69aee** resource details database migration **d38dd74** add professor + appointment migration **e434f75** Implement resource detail page and improve resource card interactions **701bb90** add appointment controller **44b2563** implement dashbord model **701bb90** add appointment controller **725b6bf** Add resource detail fields and dashboard queries	5